

Studying LLM-assisted automated test generation for Machine Learning in Python

Théo GINGUENÉ

August 31, 2026

Abstract

...

Résumé

...

Contents

1	Introduction	3
1.1	Context	3
1.2	Problem	3
1.3	Contributions	4
2	Background and related work	5
2.1	Search-based software testing	5
2.2	Automated testing of machine-learning software	5
2.3	Large language models for test generation	6
2.4	Combining LLMs and search-based test generation	6
2.5	Positioning of this work	6
3	Approach	7
3.1	New providers	7
3.2	Prompt engineering	7
3.3	Regex	9
3.4	Adding imports	9
3.5	pybind11 functions	10
3.6	Statement fixes	10
3.6.1	Asserts	10
3.6.2	Unary not	11
3.6.3	Equality between variable references	11
3.7	Evaluation	11
3.8	Compressing context	11
3.9	Generating function parameters	11
4	Evaluation	12
4.1	Dataset	12
4.2	Experiment setup	13
4.2.1	Containerization	14
4.2.2	Configuration	14
4.3	Results	14
4.3.1	Deserializer improvements	14
4.3.2	Prompt engineering and context compression	15
4.3.3	Model comparison	16
4.4	Findings	17
4.5	Threats to validity	17
4.5.1	Internal validity	17
4.5.2	External validity	18
4.5.3	Construct validity	18
4.5.4	Conclusion validity	18

5	Conclusions	18
5.1	Summary	18
5.2	Future work	19
5.3	Lessons learned	19
	Acknowledgments	19
	Keywords	19
	References	20
A	RSE de l'organisation d'accueil	21
B	Experiment results	21
B.1	Deserializer	21
B.2	Prompt comparisons	22
B.3	Model comparisons	23

1 Introduction

Software testing is an essential part of software development, as it helps detect defects and provides evidence that software behaves as intended. Historically, this has been done manually, which can be tedious and time-consuming, particularly for large and complex software systems. To address this issue, automated test generation tools have been created. Automated test generation aims to reduce this effort by automatically producing test cases that exercise the program under test. Among the different approaches to automated test generation, search-based software testing (SBST) formulates test generation as an optimization problem and uses search techniques, often evolutionary algorithms, to maximize a fitness criterion such as code coverage. Many of them were first developed for Java, because of its widespread use in research and business, and its strong typing, simplifying the generation process. One such example is Evosuite [1], which uses an evolutionary approach based on genetic algorithms to automatically generate JUnit test suites. However, some specific fields of computer science like machine learning often use Python, whose dynamic typing and highly flexible object model make the search space more difficult to explore. Using the same principles as Evosuite, Pynguin [2] was then developed to cover this gap in test generation tools for Python.

The default algorithm these two tools use is called Dynamic Many-Objective Sorting Algorithm (DynaMOSA) [3]. DynaMOSA formulates branch coverage as a many-objective optimization problem and dynamically focuses the search on uncovered targets. While it has been shown to outperform its predecessors [3], and although such approaches can generate effective test suites for many Python programs, they can struggle when reaching a coverage target requires constructing inputs satisfying complex constraints. This problem is particularly apparent in machine learning libraries, where functions often require structured inputs such as tensors, arrays, data frames, or objects satisfying implicit relationships between their values and attributes. Previous work on automated testing of machine learning libraries has shown that conventional test-generation techniques can achieve limited coverage and that providing structured input data can help reach code that is difficult to exercise through unconstrained search alone [4]. LLM-assisted approaches subsequently emerged as a complementary strategy to SBST.

Large Language Models (LLMs) offer a potential way of addressing this limitation. Unlike evolutionary search, an LLM can use source-code context, documentation, and learned programming knowledge to directly propose inputs and sequences of operations that satisfy non-trivial constraints. This makes LLMs particularly interesting as a complement to SBST: rather than replacing the search process, an LLM can provide promising test cases from which the evolutionary search can continue.

Combining LLMs and traditional approaches such as DynaMOSA has been shown to improve code coverage further than each method individually, in a new algorithm—CodaMOSA [5]. When the search reaches a coverage plateau, the algorithm asks an LLM to generate tests for uncovered parts of the program and uses these tests to redirect the search. A central challenge in this approach is the interface between the unrestricted code generated by the LLM and the restricted internal representation required by the search-based algorithm. The generated Python code therefore has to be parsed and transformed into test cases that can be manipulated by the SBST algorithm. This algorithm has been adapted in Pynguin, through the name of LLMOSA. The resulting approach is promising for machine learning libraries because it can combine the exploratory capabilities of evolutionary search with the ability of an LLM to construct complex inputs.

1.1 Context

This paper is the result of a research internship at the SNAIL [6] team of the University of Namur, in Belgium. The internship was supervised by Dr. Xavier Devroey. The SNAIL [6] team (Software Normalization Assessment and Improvement Lab) aims to help produce and identify quality software. The team is part of the Precise [7] research center (Research Center in Information System Engineering), within the NADI institute [8] (Namur Digital Institute) of the University of Namur.

1.2 Problem

Because of the random nature of genetic algorithms, DynaMOSA has trouble generating tests for machine learning (ML) libraries, for example scikit-learn or numpy [9]. Some functions require complex data structures (*e.g.* a pandas DataFrame with a column `col_y` existing), which are hard to generate through evolution only, because that would require stumbling upon the required

constraints randomly. LLMOSA seems like a promising alternative, because providing the code of the subject under test (SUT), an LLM can produce such data structures. However, when trying to run LLMOSA on code oriented toward machine learning, a few problems occurred, making the LLM unable to generate tests.

As discussed in Section 1, to combine the LLM and DynaMOSA approaches, Pynguin deserializes the generated code from the LLM (a sequence of characters) into a list of test case chromosomes, a format used by the DynaMOSA algorithm to evolve the test cases. This conversion (as described in figure 1) operates in two phases. The rewriter first takes the LLM response, extracts code blocks and specifically Python functions named `'test_...'`, and transforms statements to make them easier to deserialize (*e.g.* walrus expressions are transformed into regular assignments). The deserializer then converts the simplified code into Pynguin’s classes.

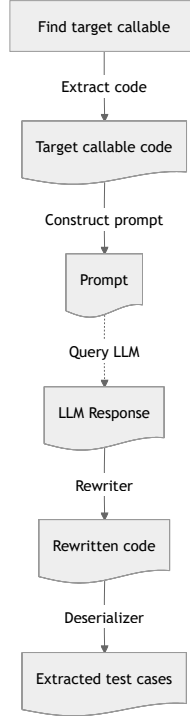


Figure 1: LLM test generation sequence

Each step adds uncertainty and potential errors. This paper aims at solving these problems one by one, first explaining them, before solving them and evaluating the impact of these changes. Our goal is to investigate Pynguin’s LLMOSA algorithm, in particular—though not only—for machine learning applications, and try to improve it. We will also try to evaluate the impact of different important parameters, such as the prompt used to generate the tests, and the model used to generate them.

1.3 Contributions

In this paper, we present the following contributions:

1. We introduce an extensible LLM-provider system and implement a local provider based on Ollama, enabling experiments with open-weight models without relying on an external API.
2. We propose an more detailed prompting and context-compression strategy designed to produce test cases that are both useful for SBST and compatible with Pynguin’s test-case representation.
3. We improve the LLM-to-test-case pipeline by resolving imports from the module under test and by extending the rewriter and deserializer to support additional Python constructs and assertions.

4. We address several practical robustness issues encountered when applying LLMOSA to machine learning-oriented Python code, including interactions with native functions such as those exposed through pybind11.
5. We evaluate these changes experimentally on modules from several machine learning-related Python libraries, measuring their effects on code coverage and execution time and comparing different LLM models.

2 Background and related work

2.1 Search-based software testing

Search-based software testing (SBST) formulates test generation as an optimization problem. The test generator searches for test cases that maximize a fitness function, usually code coverage. Evolutionary algorithms are particularly well suited for this type of tasks.

An example of this approach is EvoSuite, which automatically generates JUnit test suites for Java programs using evolutionary search [1]. EvoSuite demonstrated that search-based techniques can generate high-coverage test suites without requiring manually written tests. However, applying the same techniques to dynamically typed languages such as Python introduces additional challenges. In particular, the absence of static type information makes it more difficult to determine which objects can be used as arguments to a function and how those objects can be constructed.

Penguin was introduced to address automated unit test generation for Python [2]. It supports several search-based test-generation algorithms, including MIO, MOSA, and DynaMOSA. An empirical study found that evolutionary approaches outperform random test generation for Python programs, with DynaMOSA achieving the highest coverage among the evaluated approaches [2]. Nevertheless, the study also identified type inference and the construction of appropriate objects as fundamental limitations for automated test generation in Python.

DynaMOSA extends the many-objective optimization approach by dynamically selecting the coverage targets that should guide the search [3]. Rather than optimizing all coverage targets equally throughout the entire search, DynaMOSA focuses computational effort on targets that are currently relevant. This makes the search more efficient, but it does not fundamentally solve the problem of constructing inputs that satisfy complex constraints. If reaching a branch requires a particular combination of values, object attributes, or method calls, evolutionary mutation may have a very low probability of discovering the required sequence.

This limitation is particularly relevant to the machine-learning ecosystem. A function may require not only an object of a particular type, but also an object satisfying constraints on its structure and contents. For example, a function may require a pandas DataFrame containing a particular column, a tensor with a specific shape and dtype, or an estimator that has previously been fitted. Such constraints make the search space much harder to explore.

For a more in-depth understanding of search-based software testing (SBST), specifically for machine learning applications, we refer the reader to Chapter 2 of Lucas Berg’s thesis [4], explaining in detail the different SBST approaches, test-case generation, fuzzing, and ML-specific testing.

2.2 Automated testing of machine-learning software

Machine-learning software introduces specific testing challenges. ML libraries manipulate structured numerical data and expose APIs with constraints involving tensor shapes, data types, dimensions, model states, and relationships between multiple inputs.

Previous work has therefore investigated specialized approaches for testing machine-learning libraries. In particular, structured input generation has been proposed as a way to improve automated unit-test generation for ML-oriented Python software [4]. Instead of relying exclusively on unconstrained search to construct complex objects, structured generators can provide inputs that already satisfy part of the constraints required by the API. This allows the search algorithm to spend more of its budget exploring program behaviour rather than discovering the representation of valid inputs.

Other approaches to ML testing have focused on fuzzing and domain-specific properties. These techniques can be effective when suitable transformations or invariants can be defined, but they generally require domain-specific knowledge about the software under test. This creates a complementary problem to SBST: while search-based techniques can automatically optimize coverage, they may lack the semantic knowledge required to construct meaningful inputs.

The problem can therefore be viewed as a mismatch between two capabilities. SBST provides an effective mechanism for exploring a program once suitable test cases exist, whereas generating the suitable test cases may require knowledge of API and implicit input constraints. This observation motivates the use of techniques capable of extracting such information from source code and documentation.

2.3 Large language models for test generation

Large Language Models (LLMs) provide an alternative way of generating test cases by exploiting their ability to process source code and produce executable programs. Unlike traditional search-based techniques, an LLM can directly infer relationships between function parameters, API calls, object construction, and expected behaviour from the textual representation of a program.

Recent research has consequently investigated the use of LLMs for automated test generation. These approaches differ in how much program information is provided to the model and how generated tests are validated or repaired. A central challenge is that LLM generated tests are not necessarily executable or useful for increasing coverage. Generated tests may contain incorrect imports, invalid API calls, missing dependencies, or assertions that do not correspond to the actual behaviour of the program.

Prompt design and program context are therefore important components of LLM-based test generation. Providing more source code can give the model additional information, but also increases the context size and generation cost. Conversely, aggressively reducing the context may remove information required to construct valid inputs. Recent work on code-aware prompting has investigated how program analysis and coverage information can be incorporated into LLM prompts to guide test generation more effectively [10].

2.4 Combining LLMs and search-based test generation

Rather than replacing SBST, LLMs can be used to provide promising seeds for an existing search process. This approach combines the semantic knowledge of an LLM with the systematic exploration capabilities of evolutionary algorithms.

CodaMOSA is a representative example of this approach [5]. It integrates an LLM into a search-based test-generation process and invokes the model when the search reaches a coverage plateau. The LLM is asked to generate tests targeting under-covered parts of the program, and the resulting tests are then used to redirect the evolutionary search. In an evaluation over 486 benchmarks, CodaMOSA achieved statistically significant coverage improvements over both SBST and LLM-only baselines on substantially more benchmarks than those on which it reduced coverage [5].

The CodaMOSA results suggest that the two approaches are complementary. SBST is effective at exploiting promising regions of the search space, but can become trapped when its mutations are unlikely to discover qualitatively different inputs. LLMs can instead make larger semantic jumps by proposing new sequences of operations or structured inputs. However, this advantage comes at the cost of relying on generated source code that must be correctly interpreted by the underlying test-generation framework.

This interface is particularly important for Python because of its dynamic nature and the large number of possible runtime objects. In Pynguin’s implementation, LLM-generated tests are first rewritten and then deserialized into Pynguin’s internal test-case representation.

2.5 Positioning of this work

The existing literature establishes three relevant observations. First, SBST can generate high-coverage tests for Python, but dynamically typed code makes type inference and test-input construction difficult [2]. Second, machine-learning libraries amplify this difficulty because useful inputs frequently have structural and semantic constraints that are difficult to discover through unconstrained mutation [4]. Third, LLMs can provide useful test cases based on the code semantics, and approaches such as CodaMOSA demonstrate that combining LLM generation with SBST can improve coverage [5].

Our work focuses on the practical interface between an LLM and Pynguin’s search-based test generation. In particular, we investigate the effect of prompt design, context compression, LLM model selection, and improvements to the rewriting and deserialization pipeline. We additionally study these techniques on modules from machine-learning-related Python libraries, where structured inputs and native extensions introduce challenges that are less prominent in conventional Python benchmarks.

The objective is therefore not simply to determine whether LLMs can generate valid unit tests. Instead, we investigate which components of an LLM-assisted SBST pipeline contribute to effective coverage-oriented test generation, and under which conditions these components provide measurable benefits.

3 Approach

Before trying to improve LLMOSA, the first step was trying to run it on a code sample. We wrote an example of a code sample requiring specific input data structure (listing 1) using pandas [11], a popular library for data analysis.

Listing 1: Module under test (MUT)

```

1 import pandas as pd
2
3 def process_df(df: pd.DataFrame) -> bool:
4     '''Applies different operations to a DataFrame, in place.'''
5     df.drop(columns=['col1'], inplace=True) # Requires col1 to exist
6     return df.empty

```

This sample has two different output to be tested, **True** and **False**. The objective is to produce both compliant and non-compliant inputs to test this module. Non compliant outputs are relatively easy to generate, as any non DataFrame input will do. However, generating a compliant input is more difficult, as it requires a DataFrame with a column named `col1`, and another column, empty or not. This is an example of a function that is hard to test with DynaMOSA, but *should* be easy to test with LLMOSA.

To test this, we tried running both DynaMOSA and LLMOSA on this module, with the default configuration. However, running pynguin with LLMOSA required an OpenAI API key, which has the following drawbacks:

1. Consistency: OpenAI can discontinue access to a specific model at any time, making it difficult to reproduce experiments. Open weights models can be downloaded, stored and used at any time for a better comparison and reproducibility, providing more stable results and full control over the model.
2. Price: When benchmarking or through intensive use, cloud models token use can quickly add up to substantial expenses. Their prices are also subject to change, which may make it difficult to reproduce experiments later.

For industrial purposes, it is also important to be able to run the tests locally, without relying on an external provider. This is particularly relevant for sensitive code, where sending it to a third-party provider may not be acceptable.

3.1 New providers

To reduce this dependency to an external provider, we decided to refactor the LLM logic and add support for a new provider, Ollama. This provider can be used to run open models [12] locally, solving the issues raised above. A configuration entry was added; `-provider`, taking as possible values the enum `LLMProvider`—currently `OPENAI` or `OLLAMA`.

3.2 Prompt engineering

Due to the constraints of the rewriter, we noticed that the LLM often failed to provide an appropriate structure. The prompt was minimalist and did not provide strong guidance to the LLM (listing 2).

Listing 2: Original prompt

```

Write unit tests for the following callables that Pynguin failed to cover:
{callables_section}
Module path: `{self.module_path}`
Module source code: `{self.module_code}`

```

To try and guide the LLM towards providing a correctly structured response, we listed the requirements to satisfy for the rewriter to work:

1. The answer should be inside a code block:
Pynguin uses the regex `r"```python([\s\S]+?)(?:```|$)"` to extract the code block from the LLM response. If the answer is not inside a code block annotated with triple backticks and "python", it will not be parsed. If no code block is found, the rewriter will return an empty list of test cases, which will make the algorithm fail to generate any test case.
2. Each function must be named `'test_...'`:
The rewriter specifically checks if the function name starts with `'test_'`. If it does not, the function will be ignored and not parsed. It also has to either be top-level or inside a top-level class, but not inside another function or nested class.
3. Each function must have no arguments:
While the rewriter can handle functions with arguments, the deserializer cannot, due to the fact that Pynguin's test case representation does not support arguments. If a function has arguments, it will be ignored and not parsed. Without changing the whole test case representation, this requirement cannot be lifted.
4. Pytest features (*e.g.* fixtures) are not supported:
Again, due to the limitations of Pynguin's test case representation, the deserializer cannot handle pytest features. Decorators in general are not rewritten, and will be ignored (except for `@pytest.mark.parametrize`, as explained below).

We added a bit of context, and complementary instructions such as:

1. Not rewriting the SUT's code, for three main reasons: first, it would skew pynguin's code coverage results, as the tests would not run on the SUT's actual code but on the rewritten version; second, it would make the tests less maintainable, as they would have to be updated if the SUT changes; third, it saves tokens and run time, which is beneficial for the overall efficiency of the testing process.
2. Following the Arrange, Act, Assert pattern, which is a common pattern in unit testing. It helps to structure the tests and makes them easier to read and understand, which is useful for debugging and guides the LLM toward generating higher quality tests.

All these instructions were compiled into a new prompt (listing 3), with an emphasis on the fact that the functions should have no arguments, because this was the main cause of parsing failures, and an example of a valid test function. To help with running the same test with different values, we also added support for `pytest.parametrize`. We rewrite the decorated function as one function per parameter, with the parameter initialized at the start of the function body. This allows the LLM to use `pytest.parametrize` to generate multiple tests with different values, while still satisfying the requirement of having no arguments.

Listing 3: Our prompt proposition

```
You are writing tests to be used as seed for a SBST algorithm. Your goal is to
→ improve as much as possible the coverage of the tests.
Write unit tests for the following callables that Pynguin failed to cover:
{callables_section}
Module path: `{self.module_path}`
Module source code:
```python
{compressed_module_code}
```

You answer will be parsed for mutations, so here are the guidelines you need to
→ follow:
- Answer in a code block only using; one function for each test case, with NO
→ ARGUMENTS, NO HELPER FUNCTIONS AND NO CLASSES.
- If needed, instantiate vars in the body of the test func or use
→ pytest.parametrize, but DO NOT USE ANY OTHER PYTEST FEATURE (e.g. DO NOT USE
→ FIXTURES), as that will make the parsing fail.
```

```

- Do not rewrite the SUT's code in the tests. If you want for example to call a
  ↳ function or instantiate a class, import it.
- Think first then answer in simple, concise assertion tests, split in small
  ↳ functions. Follow the Arrange, Act, Assert pattern.

Here are some examples; *NEVER DO*:
def func_all_tests(param):
    var0 = param
    ...

*INSTEAD DO*:
I need to test the function `func` that takes a parameter and returns a value. I
  ↳ will test the following scenarios: ...
```python
from module_to_test import func
def test_feat1():
 var0 = ...
 var1 = func(var0)
 assert ...
```

REMEMBER: NO ARGUMENTS IN YOUR TEST FUNCTIONS (if you don't use
  ↳ pytest.parametrize)"

```

Naturally, the question that arises is the following:

RQ1: *Can our prompt improve the algorithm performance?*

Our prompt uses more tokens than the original one, however it is more specific and provides more guidance to the LLM. It is therefore expected to improve the algorithm performance, but it is not guaranteed. We will try to find out and answer this question in section 4.

3.3 Regex

As mentioned above, the rewriter uses a regex (`r"```python([\s\S]+?)(?:```|`$)"`) to extract the code block from the LLM response. We modified this regex to make it more robust, and to allow for more variations of the code block. The new regex is `r"```py(?:thon)?([\s\S]+?)(?:```|`$)"`, which allows for the code block to be annotated with both “python” and “py”, as both are considered correct, and the LLM often uses the latter.

3.4 Adding imports

After adding the new prompt, we were able to consistently generate correct rewritten tests. When deserializing though, lots of statements got lost in the process: function calls, some asserts, attributes, some comparisons, *etc.* To illustrate this, we show a sample of the rewritten output (listing 4) and the deserializer result (listing 5).

Listing 4: Rewritten output (sample)

```

1 def test_process_df_with_missing_values():
2     var_0 = 'Test for a DataFrame with missing values.'
3     var_1 = 'col1'
4     var_2 = 'col2'
5     var_3 = 1
6     var_4 = 2
7     var_5 = None
8     var_6 = [var_3, var_4, var_5]
9     var_7 = 3
10    var_8 = [var_3, var_4, var_7]
11    var_9 = {var_1: var_6, var_2: var_8}
12    df = pd.DataFrame(var_9)
13    var_10 = process_df(df)

```

```

14     var_11 = True
15     var_12 = var_10 is var_11
16     assert var_12

```

The deserializer result is missing many statements from the rewritten output, which is a problem for the algorithm’s performance. The most important missing statements are the function calls, which are the main part of the test case. Without them, the test case is useless as it does not execute a single line of code from the SUT.

Listing 5: Deserializer result

```

1  def test_generated_function():
2      str_0 = 'Test for a DataFrame with missing values.'
3      str_1 = 'col1'
4      str_2 = 'col2'
5      int_0 = 1
6      int_1 = 2
7      none_type_0 = None
8      list_0 = [int_0, int_1, none_type_0]
9      int_2 = 3
10     list_1 = [int_0, int_1, int_2]
11     dict_0 = {str_1: list_0, str_2: list_1}
12     bool_0 = True

```

After further investigation, we figured that the main cause of parsing failures was the impossibility to resolve objects from the libraries used by the test suite, including from the MUT (*e.g.* `pd.DataFrame`, `process_df`). Each unparsed statement then in turn caused the parsing of the subsequent statements to fail. Two options were considered:

1. Gather the imports written by the LLM, import the corresponding libraries and search for matches in the imported objects.
2. Search in the libraries imported from the module under test (MUT).

Option 1 makes sure that libraries explicitly cited by the LLM are imported, but has other implications, such as making sure that the package is installed, performance issues, and more unpredictable behavior. That is why we opted for option 2. This choice is further justified by the fact that most of the time, it is not required to import libraries other than those used by the MUT. We still added the imports hinted by the LLM in the rewriter, both for debugging purposes and easing the implementation of option 1 if necessary.

3.5 pybind11 functions

pybind11 [13] is a library that allows to create Python bindings for C++ code. It is used in many machine learning libraries, such as PyTorch, to expose C++ functions to Python. However, pybind11 functions are not regular Python functions, and would make penguin crash when trying to retrieve their AST, because they reference an unreachable module. This was fixed by setting the module tree to `None` when the module is not found.

3.6 Statement fixes

Even after adding the imports, even though the majority of statements were deserialized correctly, some still failed to be deserialized. We identified a few causes:

3.6.1 Asserts

Asserts were not added to the test case due to an incorrect configuration. If the assertion generator was not set to `AssertionGenerator.LLM`, the deserializer would not add asserts to the test case. However, the LLM was still generating asserts, which were then ignored by the deserializer. This was fixed by simply always adding asserts to the test case.

3.6.2 Unary not

Unary not statements (*i.e.* `a = not b`) were not deserialized because the deserializer was not handling them. We added a special case checking for the not operator and creating the appropriate `ASTAssignStatement`. This single change improves the deserializer success rate significantly because the LLM often uses the not operator in its generated code, especially when asserting that a boolean value is False (*e.g.* `assert not df.empty`).

3.6.3 Equality between variable references

A problem in the implementation of the deserializer (when testing equality between variable references, *e.g.* `z = x == y`) was causing it to crash on some occurrences, stopping completely the test generation. The problem was later found and a patch was added in the main branch of Pynguin by the maintainer [14]. Therefore, we did not include this problem fix in our comparison, as it is already available in the main branch of Pynguin.

3.7 Evaluation

After implementing all the changes above, we ask ourselves to which extent the changes impact the algorithm performance. We will try to answer the following research question:

RQ2: *Is our new generation approach more effective?*

The evaluation will be done in section 4.

3.8 Compressing context

The fixed deserializer was able to parse the rewritten output correctly, but when testing LLMOSA on a large module, we ran into issues. The LLM was often unable to generate a proper response. Sometimes it would time out, while other times it would a response that was not parsable by the rewriter or deserializer or simply unusable. The run time also increased significantly, as the LLM had to process a lot of information. We identified that the main cause of this problem was the context size. The LLM was given too much information—the entire module code, which could be very large, and the list of callables to cover, which could also be large (often the entire module). To solve this problem, we implemented a context compression algorithm.

Using the module’s AST, we can identify the callables we want to cover, and extract only the relevant code for each callable (*e.g.* the class the callable belongs to). The other callables are then removed from the prompt (and replaced with a “# [...]” comment to hint that they were removed). This way, the LLM is given only the relevant information for each callable. We also limited the number of callables to cover in each prompt, to avoid giving too much information to the LLM.

We consider this compression algorithm as part of the prompt engineering, as it is a way to improve the prompt by reducing the context size. As such, we will evaluate its impact on the algorithm performance in section 4 to try and answer RQ1.

3.9 Generating function parameters

One of the main issues with SBST is the difficulty of randomly stumbling across valid function parameters. To adress this issue, we implemented a system to generate function parameters using an LLM. The LLM is given the function signature and docstring, and is asked to generate valid parameters for the function. The generated parameters are then used to call the function in the test case.

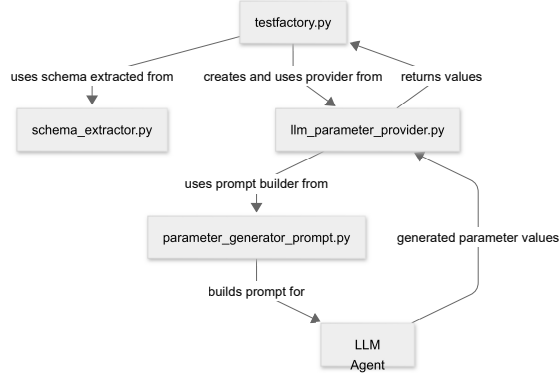


Figure 2: Parameter generation flow

Due to the fact that this system is still in development and the time constraints, this feature was not evaluated.

4 Evaluation

To evaluate the impact of our changes on the algorithm performance, we ran a series of experiments on a set of modules from popular machine learning libraries. We compared the original approach with our new approach, and the original prompt with our new prompt, as well as comparing different models. The experiments were run on a compute cluster using Kubernetes, with each experiment running in a separate pod. Each experiment was run 30 times to account for the randomness of the algorithm, and the results were collected and analyzed.

We used the Mann–Whitney U test because each experiment contains independent runs of a randomized algorithm, and we report the Vargha–Delaney effect size $\hat{A}_{12}$ in addition to the p -value, according to the recommendations of Arcuri and Briand [15]. An effect size close to 0.5 indicates that the two configurations perform similarly. In all comparisons, $\hat{A}_{12}$ is oriented so that values above 0.5 favor the left-hand configuration: for coverage score this means a larger score, while for duration the values are reversed, so it means a shorter execution time. Values below 0.5 therefore favor the right-hand configuration.

When we later refer to the “score”, we mean the coverage score, which is the main metric used to evaluate the performance of the algorithm. The coverage score is a value between 0 and 1, representing the percentage of code covered by the generated tests. A higher score indicates better performance. While it is not a perfect representation of a test suite’s quality, it is a good indicator of the algorithm’s performance and one of the easiest metrics to measure. It is also the main metric used in most papers about automated test generation.

4.1 Dataset

Considering the goal of this paper is to improve LLMOSA for machine learning applications, we selected a few modules from popular machine learning libraries. We selected modules from pytorch, TabGAN, and transformers, which are popular libraries for deep learning and natural language processing. They were selected randomly between libraries that had a wide range of complexity and popularity, and that covered different aspects of machine learning, such as neural networks, tabular data generation, and natural language processing. All these libraries are part of the Awesome Python list [16], which is a curated list of Python frameworks, libraries, tools, and resources. The modules were selected randomly from the libraries, to avoid any bias in the selection process. Here are the exact modules we used in our benchmark, with their path in the library:

- `pytorch/torch/_numpy/_ndarray.py`
- `pytorch/torch/_numpy/_getlimits.py`
- `pytorch/torch/nn/attention/bias.py`
- `pytorch/torch/nn/attention/flex_attention.py`
- `Tabular-data-generation/src/_ForestDiffusion/diffusion_with_trees_class.py`

- `Tabular-data-generation/src/tabgan/adversarial_model.py`
- `Tabular-data-generation/src/_ctgan/transformer.py`
- `transformers/src/transformers/tokenization_utils_base.py`
- `transformers/src/transformers/pipelines/text_generation.py`
- `transformers/src/transformers/data/metrics/squad_metrics.py`
- `pytorch/torch/nn/modules/rnn.py`
- `pytorch/torch/_numpy/_util.py`
- `pytorch/torch/_numpy/_funcs.py`
- `pytorch/torch/_numpy/_ufuncs.py`
- `pytorch/torch/_numpy/_normalizations.py`

4.2 Experiment setup

We want to answer the following research questions:

RQ1: *Can our prompt improve the algorithm performance?*

RQ2: *Is our new generation approach more effective?*

To answer RQ2, we ran the experiments with the original approach and our new approach, both with the same configuration and using LLMOSA. We called this experiment “classic”.

To answer RQ1, we ran the experiments with the default prompt, our new prompt with context compression (“UncoveredTargetsPrompt”), and the same new prompt without context compression (“NoCompressionPrompt”), again keeping the remaining configuration unchanged. We called this experiment “prompt”.

We also introduce a new question, which is not directly related to our changes, but is interesting to investigate:

RQ3: *Which model(s) performs best?*

To answer RQ3, we ran the experiments with the same configuration and using LLMOSA, but with different models. We compared the performance of the following models, chosen to vary in size, specific capabilities, and model family:

- `qwen2.5:1.5b`
- `qwen2.5:3b`
- `qwen2.5-coder:0.5b`
- `qwen2.5-coder:1.5b`
- `qwen2.5-coder:3b`
- `qwen2.5-coder:7b`
- `qwen3:4b`
- `qwen3.5:4b`
- `gemma4:e2b`
- `gemma4:12b`
- `gemma4:26b`
- `gemma4:31b`

4.2.1 Containerization

We deployed our experiments in a compute cluster using Kubernetes. A job was created for all experiments, which created a pod for each experiment. Each pod was running a container, which contained all the dependencies required to run the experiments. The container was built using a Dockerfile, which installed all the dependencies and copied the source code of our project into the container. The Dockerfile was optimized to improve the build time, as the installation of the dependencies was the most time consuming part of the build process.

4.2.2 Configuration

As per the recommendations of Arcuri and Briand [15] on statistical tests for assessing randomized algorithms, and because of the time needed to run the benchmark (up to a few hours per experiment), we ran each experiment 30 times on each one of the 15 modules to account for the randomness of the algorithm. The configuration of the experiments was the default configuration of Pynguin, with the following changes:

- provider: Ollama
- algorithm: LLMOSA
- model-name: See for each experiment
- call-llm-for-uncovered-callables: True
- call-llm-on-stall-detection: True
- max-plateau-len: 5

4.3 Results

4.3.1 Deserializer improvements

We compared the original baseline approach (“base”) with our new deserializer (“ours”), which includes import resolution, extended deserializer support for additional Python constructs, and fixes for robustness issues. Figure 3 shows the distribution of coverage scores across the tested modules. The corresponding table is available in the appendix (Table 1).

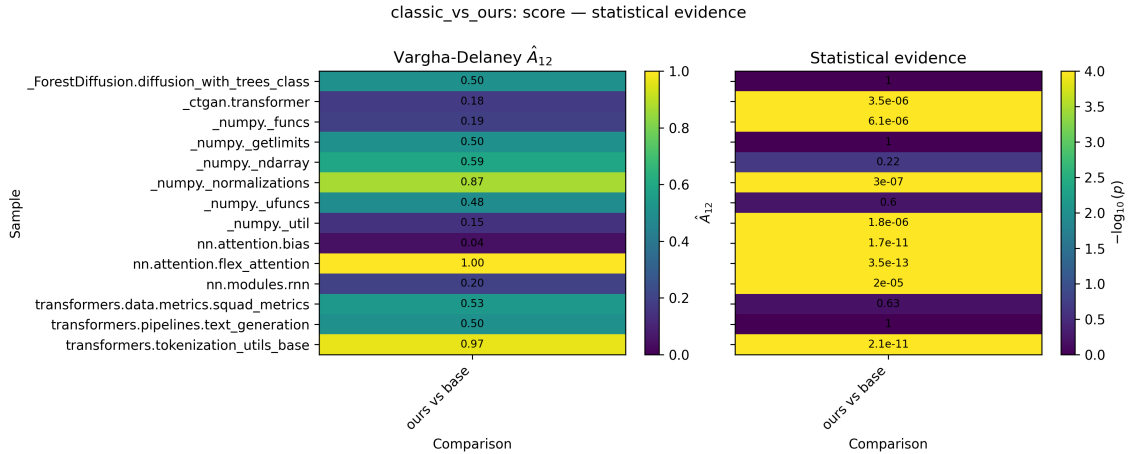


Figure 3: Distribution of coverage scores (classic baseline vs. new deserializer)

The results reveal a more nuanced picture than initially expected. On coverage scores, the baseline approach performs better on 6 out of 15 modules, while our new deserializer performs better on 5 modules, with 3 modules showing no significant difference. Of the 8 modules with statistically significant differences ($p < 0.05$), the baseline is superior on 5. Notably, our approach achieves large improvements on **nn.attention.flex_attention** ($\hat{A}_{12} = 1.000$, large effect) and **transformers.tokenization_utils_base** ($\hat{A}_{12} = 0.967$, large effect). However, the baseline performs substantially better on modules such as **nn.attention.bias** ($\hat{A}_{12} = 0.045$, $p < 0.001$, large effect) and **numpy._util** ($\hat{A}_{12} = 0.146$, $p < 0.001$, large effect).

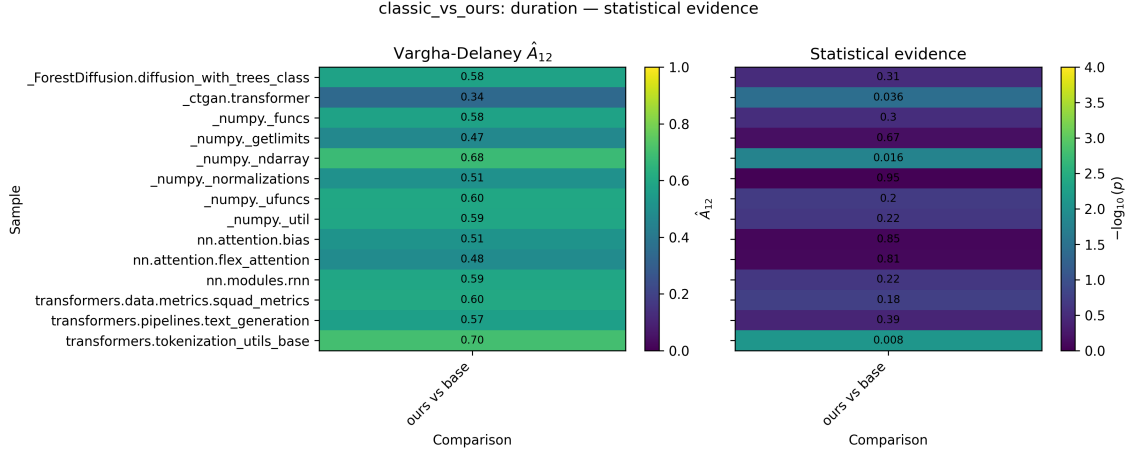


Figure 4: Statistical evidence for deserializer comparison (duration)

Results for execution time tend to favor our new deserializer, with 12 out of 15 modules showing faster execution times. However, the statistical significance of these differences is limited.

These mixed results suggest that while our improvements are beneficial for some modules (particularly those requiring complex test case construction), they do not uniformly outperform the baseline across all tested libraries. The improvements show their greatest benefit on modules with significant parsing demands, while the baseline approach remains more effective on others. This indicates that the effectiveness of deserialization improvements is heavily context-dependent and may require further investigation into which specific code patterns benefit most from our enhancements.

4.3.2 Prompt engineering and context compression

The new prompt (“UncoveredTargetsPrompt”) includes detailed instructions and context compression. We compare it both with the default prompt (“DefaultPrompt”) and with the same instructions without compression (“NoCompressionPrompt”), to measure their relative performance. Figure 5 shows the distribution of coverage scores for all three prompt strategies across the tested modules.

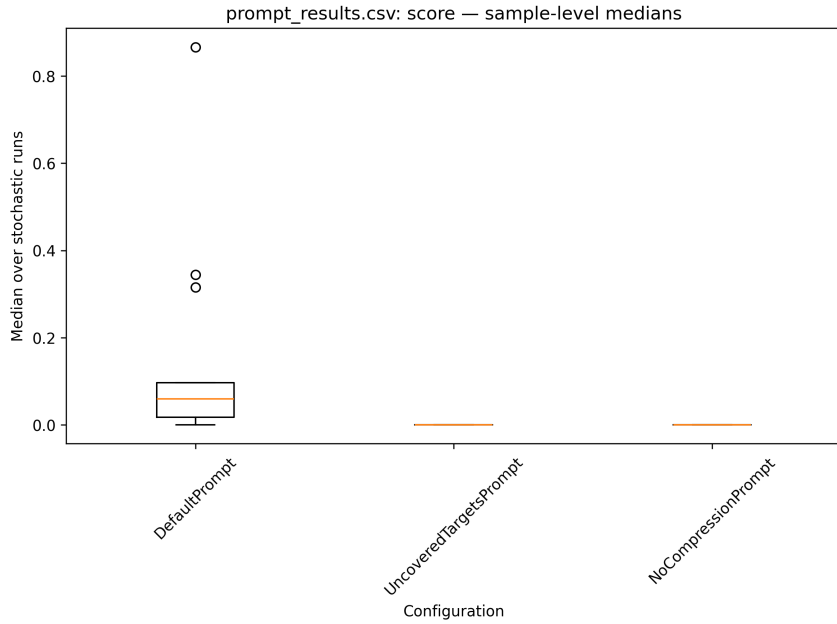


Figure 5: Distribution of coverage scores across different prompt strategies

Contrary to expectations, the results demonstrate that the default prompt (“DefaultPrompt”) substantially outperforms the detailed prompt (“UncoveredTargetsPrompt” and “NoCompressionPrompt”) on most tested modules.

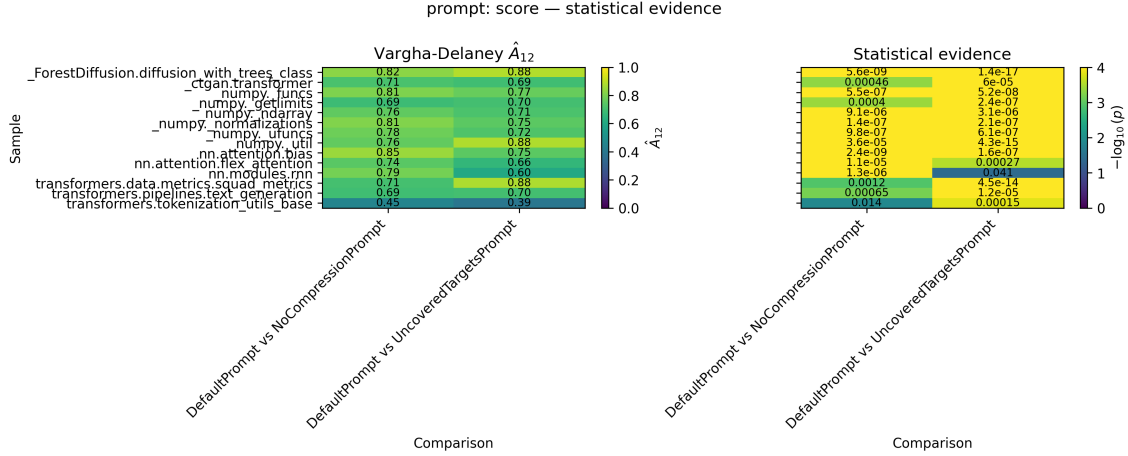


Figure 6: Statistical evidence for prompt comparison (score coverage)

Regarding execution time, the compressed prompt can be compared directly with NoCompressionPrompt to isolate the cost of context compression, while the DefaultPrompt comparison captures the combined effect of changing the instructions and context. With NoCompressionPrompt as the left-hand configuration, the direct comparison gives a score $\hat{A}_{12}$ of 0.475 on average (median 0.451) across the 14 modules. The compressed prompt has the higher score in 9 modules, but only 3 of the 5 statistically significant comparisons favor compression; the other 2 favor no compression. Thus, compression alone does not produce a consistent coverage advantage. It does, however, substantially reduce execution time: compression is faster in 13 of 14 modules according to the $\hat{A}_{12}$ direction, with 7 statistically significant differences. The complete statistical output (Table 2) reports both metrics for every sample.

These results do not support the hypothesis posed in RQ1. The default prompt remains substantially better for coverage than both alternatives, while compression mainly provides a runtime benefit. Comparing NoCompressionPrompt with UncoveredTargetsPrompt also shows that the loss in coverage cannot be attributed to context compression alone; and the detailed prompt reduces coverage on this benchmark.

4.3.3 Model comparison

To understand the effect of different language models on test generation performance, we compared multiple model variants across different model families and sizes. Figure 8 shows the distribution of coverage scores for different models.

The complete statistical output (Table 3 and Table 4) provides a detailed view of the performance differences between the various model variants.

Model selection shows mixed effects on coverage performance. The 260 weight comparisons consist of 130 duration and 130 score comparisons. For coverage, only 8 comparisons have a large effect, and the median $\hat{A}_{12}$ is 0.500, indicating no general advantage for smaller or larger weights. In contrast, duration has 44 large effects, with a median $\hat{A}_{12}$ of 0.642 because the smaller model is always the left-hand configuration. The strongest and most consistent duration differences favor the smaller model, including Gemma 4 0.5b versus 12b on **transformers.data.metrics.squad_metrics** ($\hat{A}_{12} = 0.968$, large effect), where the 0.5b model executes much faster.

We also separately compared coder and non-coder Qwen models at matched weights. Across 20 comparisons (10 per metric), coder models show essentially no advantage in duration (median $\hat{A}_{12} = 0.526$) and no advantage in coverage (median $\hat{A}_{12} = 0.489$). The direction is not stable across weights: the 1.5b coder model has higher coverage on 8 of 10 modules, whereas the 3b coder model has higher coverage on only 4. Only 2 duration and 5 score comparisons are statistically significant, and none has a large effect. These results provide no evidence that the coder variant consistently improves coverage, nor does it show a consistent disadvantage.

The results therefore reveal a more specific trade-off than a simple model-size effect. Weight seems to have a relationship with duration, but little systematic relationship with coverage. Coder tuning has no consistent effect on either metric in this dataset. Smaller models may consequently be preferred for more efficient resource usage (time, computational or financial).

4.4 Findings

Our experimental evaluation reveals unexpected results that contradict our initial hypotheses:

Regarding RQ2 (Deserializer improvements)

The improvements to the LLM-to-test-case pipeline show mixed effectiveness. While our modifications enable the deserializer to handle additional Python constructs and improve robustness for certain edge cases (particularly `nn.attention.flex_attention` and `transformers.tokenization_utils_base`), the baseline approach actually performs better on 6 out of 15 modules. It seems that while the improvements are beneficial for some modules, they do not consistently outperform the baseline across all tested libraries. This suggests that the effectiveness of deserialization improvements is context-dependent and may require further investigation into which specific code patterns benefit most from our enhancements.

Overall, in this dataset, we cannot conclude that our improvements to the deserializer consistently improve coverage. This indicates that while our enhancements are valuable for certain scenarios, they do not universally enhance test generation performance.

Regarding RQ1 (Prompt engineering and context compression)

Contrary to expectations, the default prompt substantially outperforms the more detailed on most metric-sample comparisons. The additional “UncoveredTargetsPrompt” versus “NoCompression-Prompt” comparison allows the effects of context compression to be assessed separately from the effects of the revised instructions. Taken together, the results indicate that, in our specific case, giving more detail and guidance to the LLM actually *reduces* coverage performance. The default prompt is simpler and more effective, while context compression mainly provides a runtime benefit without improving coverage. This experiment, however, was run only with the qwen2.5-coder:0.5b model (one of the lightest), and it is possible that other models may respond differently to the prompt changes. Further evaluation with different models would be needed to confirm the generality of this finding. We can conclude, however, that giving more detailed instructions to the LLM does not guarantee better performance, and that prompt design is a complex task that requires careful empirical evaluation.

Regarding RQ3 (Model selection)

Model choice has limited systematic impact on coverage quality across our benchmark. Among the 260 weight comparisons, only 8 show large coverage effects and the median coverage $\hat{A}_{12}$ is 0.500. Weight does affect duration: 44 comparisons have large duration effects and the median duration $\hat{A}_{12}$ is 0.642 in favor of the smaller, left-hand model, although the 12b–26b Gemma comparison shows that this relationship is not strictly monotonic. Coder versus non-coder comparisons are similarly inconclusive: coder models have no large effects and show medians of 0.526 for duration and 0.489 for coverage. Smaller models can therefore be useful for reducing inference time, but neither size nor coder tuning consistently improves coverage.

Implications

These results highlight the importance of empirical evaluation in LLM-assisted test generation research. Our intuitions about what would improve coverage—more detailed prompts, extended deserializer support, larger models—were not consistently validated. This suggests that LLM-assisted SBST is more complex than initially understood, with non-obvious interactions between components. The baseline approach, despite being simpler, proves to be more robust across diverse code characteristics. Future work should focus on understanding why the baseline approach is effective and identifying which specific code patterns would benefit from our enhancements, rather than assuming that more sophisticated approaches are universally better.

4.5 Threats to validity

Several limitations of this study should be noted:

4.5.1 Internal validity

The experimental setup uses 30 independent runs per configuration to account for the randomness inherent in both LLMOSA and the local LLM inference. However, the computational costs may

introduce variability across different environments. The experiments were conducted on a compute cluster using Kubernetes, which provides isolation but does not completely eliminate environmental variations, such as hardware differences. Additionally, the choice of statistical test (Mann-Whitney U) assumes independence of observations.

4.5.2 External validity

Our evaluation focuses on 15 modules from three machine learning libraries (PyTorch, transformers, and TabGAN). While these represent popular and diverse libraries, the generalizability to other Python projects, particularly non-machine-learning codebases, remains uncertain. The modules were selected randomly to avoid bias, but the subset may not be fully representative of the broader ecosystem. Furthermore, the improvements to import resolution and deserializer robustness are tailored to handle specific patterns observed in machine learning libraries, and their effectiveness on other types of Python code is unknown.

4.5.3 Construct validity

We measure test generation effectiveness through two metrics: code coverage (measured as a score) and execution time. While these are standard metrics in the SBST literature and align with our research objectives, they do not fully capture test quality. Metrics such as fault detection ability or test maintainability are not evaluated. The coverage metric is provided by Pynguin’s internal implementation and may be influenced by the specific instrumentation and branch coverage definitions used.

4.5.4 Conclusion validity

Statistical significance was determined using a the Mann-Whitney U test without correction for multiple comparisons. While this follows the recommendations of Arcuri and Briand [15] for per-sample comparisons, the lack of multiple comparison correction increases the error rate across all comparisons.

5 Conclusions

5.1 Summary

This paper investigates practical improvements to LLM-assisted test generation for machine learning libraries in Python, focusing on the interface between Large Language Models and the SBST algorithm LLMOSA in Pynguin. We implemented three key modifications: (1) an extensible LLM provider system enabling local, open-weight models through Ollama; (2) enhancements to the rewriter and deserializer, including improved import resolution and support for additional Python constructs; and (3) a more detailed prompt with context compression designed to guide LLM test generation.

Experimental evaluation on 15 modules from popular machine learning libraries reveals surprising findings that challenge common assumptions about LLM-assisted test generation. Contrary to expectations, the simpler default prompt substantially outperforms the enhanced prompt with detailed guidance (93% of comparisons favor the default prompt), and our deserializer shows mixed results, showing no significant difference in most cases. Model selection has minimal impact on coverage, with larger models offering no significant advantage over smaller models, only increased inference latency.

These results highlight the importance of empirical evaluation and suggest that LLM-assisted SBST is more nuanced than initially anticipated. Rather than confirming that more sophisticated approaches uniformly improve performance, our evaluation demonstrates that the baseline approach, despite being simpler, provides better robustness across diverse code characteristics.

Due to not having substantial drops in coverage, smaller models may be more practical for real-world applications, offering comparable coverage with significantly lower computational or financial resource consumption.

The findings suggest that simpler approaches seem to integrate more effectively with the SBST algorithm, pointing toward the need for careful investigation into which specific project characteristics benefit from targeted improvements, rather than applying complex modifications broadly.

5.2 Future work

Several promising directions for future work emerge from this study:

1. **Parameter generation:** The parameter generation system (described in section 3) remains incomplete. Finishing development and evaluating this component could further improve LLMOSA’s ability to construct complex inputs required by machine learning APIs.
2. **More extensive evaluation:** Evaluating the approach on additional machine learning libraries and domains would strengthen our analysis and identify library-specific improvements. The current evaluation is limited to 15 modules from three libraries, and a broader dataset could reveal patterns not captured in this study.
3. **Hybrid approaches:** Investigating deeper integration between LLM-generated tests and the search algorithm, such as using the LLM to dynamically adjust search parameters or guide mutation strategies, could further improve coverage.
4. **Cost-benefit analysis:** A more detailed analysis of the trade-offs between model size, inference cost, and coverage improvement could help developers select appropriate models for their use cases.

5.3 Lessons learned

Throughout this project, I sharpened my Python skills, improved my knowledge of Docker, especially how to optimize Dockerfiles for improved build speed, and learned how to use Kubernetes to deploy and manage containers in a compute cluster. I followed the rigor and methodology of scientific research, reviewing the state of the art, identifying a problem, proposing a solution, and evaluating it. I had to conduct experiments in a reproducible manner, with proper documentation, replication, and statistical analysis. I also learnt how to write a scientific paper, and how to present my work clearly and concisely. I was also able to attend a few presentations and thesis defenses, which gave me a better understanding of the research process and the challenges faced by researchers. Finally, I improved my time management, prioritization skills and autonomy, which is crucial in research. These lessons will be useful for my future career, as they are applicable to any research or development project alike.

Acknowledgments

I would like to thank Xavier DEVROEY (my tutor) and Lucas BERG for providing valuable guidance during my internship.

Thanks as well to Paul TEMPLE, and the whole SNAIL team for their support and feedback during the development of this work.

Keywords

Software Testing, Search-Based Software Testing (SBST), Large Language Models (LLMs), Python, Pynquin, Ollama

References

- [1] Gordon Fraser and Andrea Arcuri. “EvoSuite: automatic test suite generation for object-oriented software”. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ESEC/FSE ’11. Szeged, Hungary: ACM, 2011, pp. 416–419. ISBN: 978-1-4503-0443-6. DOI: 10.1145/2025113.2025179. URL: <http://doi.acm.org/10.1145/2025113.2025179>.
- [2] Stephan Lukasczyk and Gordon Fraser. “Pynguin: Automated Unit Test Generation for Python”. In: 44th International Conference on Software Engineering Companion (ICSE ’22 Companion). 2022. DOI: 10.1145/3510454.3516829.
- [3] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. “Automated Test Case Generation as a Many-Objective Optimisation Problem with Dynamic Selection of the Targets”. In: *IEEE Transactions on Software Engineering* 44.2 (2018), pp. 122–158. DOI: 10.1109/TSE.2017.2663435.
- [4] Lucas Berg. “Improving automated unit test generation for machine learning libraries using structured input data”. Master’s thesis. University of Namur, June 17, 2024.
- [5] Caroline Lemieux et al. “CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2023, pp. 919–931. DOI: 10.1109/ICSE48619.2023.00085.
- [6] *SNAIL Team*. 2026. URL: <https://snail.info.unamur.be/about/>.
- [7] *Precise Research Center*. 2026. URL: <https://nadi.unamur.be/research-centers/precise>.
- [8] *NADI Institute*. 2026. URL: <https://nadi.unamur.be>.
- [9] Nicolas Erni et al. *SBFT Tool Competition 2024 – Python Test Case Generation Track*. 2024. arXiv: 2401.15189 [cs.SE]. URL: <https://arxiv.org/abs/2401.15189>.
- [10] Gabriel Ryan et al. *Code-Aware Prompting: A study of Coverage Guided Test Generation in Regression Setting using LLM*. 2024. arXiv: 2402.00097 [cs.SE]. URL: <https://arxiv.org/abs/2402.00097>.
- [11] The pandas development team. *pandas-dev/pandas: Pandas*. DOI: 10.5281/zenodo.3509134. URL: <https://github.com/pandas-dev/pandas>.
- [12] Ollama. *Ollama*. 2026. URL: <https://github.com/ollama/ollama>.
- [13] pybind11 contributors. *pybind11*. 2026. URL: <https://github.com/pybind/pybind11>.
- [14] Lukas Krodinger. *Commit 55c6806*. June 30, 2026. URL: <https://github.com/se2p/pynguin/commit/55c680653adc216ef4130eef6411814b7d67f4e5>.
- [15] Andrea Arcuri and Lionel Briand. “A practical guide for using statistical tests to assess randomized algorithms in software engineering”. In: *Proceedings of the 33rd International Conference on Software Engineering*. ICSE ’11. Waikiki, Honolulu, HI, USA: Association for Computing Machinery, 2011, pp. 1–10. ISBN: 9781450304450. DOI: 10.1145/1985793.1985795. URL: <https://doi.org/10.1145/1985793.1985795>.
- [16] vinta. *Awesome Python*. July 3, 2026. URL: <https://github.com/vinta/awesome-python>.

Appendix A RSE de l'organisation d'accueil

| Stratégie RSE de l'organisation d'accueil (Université de Namur – faculté d'informatique) | | |
|--|---|---|
| Catégorisation de l'axe de RSE | Intitulé de l'action mise en œuvre au sein de l'organisation | Objectif poursuivi par l'organisation |
| Gouvernance | Intégration de la responsabilité et de l'éthique dans la formation des informaticiens | Former des informaticiens capables de prendre en compte les conséquences de leurs choix techniques et de leurs décisions professionnelles sur la société |
| | Développement d'une politique institutionnelle en matière d'intelligence artificielle responsable | Garantir que le développement et l'utilisation de l'IA tiennent compte des enjeux RSE tout en favorisant l'innovation |
| Relations et conditions de travail | Dispositif de prévention et de lutte contre le harcèlement PHARE | Garantir un environnement d'étude et de travail respectueux et sécurisé et prévenir les situations de harcèlement |
| | Politique d'égalité des genres, de diversité et d'inclusion | Garantir à chaque membre de la communauté universitaire un environnement inclusif et équitable |
| | Politique globale de bien-être et de sécurité | Créer des conditions favorables à l'apprentissage, à la recherche et au travail en général |
| Les bonnes pratiques des affaires | Code de bonne conduite | Fixer un cadre afin de garantir des relations respectueuses entre les différentes parties prenantes de l'Université |
| Questions relatives aux consommateurs/clients | Sensibilisation à la cybersécurité et protection des données personnelles | Former des professionnels capables de concevoir et de maintenir des systèmes numériques sûrs |
| Environnement | Tri et réduction des déchets sur le campus (flots de tri), achats plus responsables | Favoriser le recyclage et diminuer l'impact environnemental des activités universitaires |
| | Plan de déplacement d'entreprise et promotion de la mobilité durable | Encourager les membres du personnel et les étudiants à privilégier les transports collectifs et la mobilité durable |
| Engagement sociétal | Cours « Engagement citoyen » au sein de la Faculté d'informatique | Développer la conscience citoyenne des futurs informaticiens |
| | Mise à disposition de l'expertise informatique au service de la société | Utiliser les compétences scientifiques et technologiques de la Faculté pour répondre à des problématiques concrètes de société et contribuer à une évolution responsable du numérique |
| | Collaboration avec les entreprises, associations et autres acteurs de la société | Favoriser le transfert de compétences et de savoirs, répondre à des besoins réels |

Figure 7: Stratégie RSE de l'université de Namur et de la faculté d'informatique

Appendix B Experiment results

This appendix provides detailed results for all statistical comparisons conducted during the evaluation. The tables present comparisons using the Mann-Whitney U test's p -values with Vargha-Delaney $\hat{A}_{12}$ effect sizes.

B.1 Deserializer

Table 1: All statistical comparisons: classic vs ours

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|----------|---|------|------|--------|----------------|------------|
| duration | _ForestDiffusion.diffusion_with_trees_class | ours | base | 0.3112 | 0.5767 | small |
| duration | _ctgan.transformer | ours | base | 0.0364 | 0.3422 | medium |
| duration | _numpy._funcs | ours | base | 0.3042 | 0.5778 | small |
| duration | _numpy._getlimits | ours | base | 0.6735 | 0.4678 | negligible |
| duration | _numpy._ndarray | ours | base | 0.0163 | 0.6811 | medium |
| duration | _numpy._normalizations | ours | base | 0.9470 | 0.5056 | negligible |
| duration | _numpy._ufuncs | ours | base | 0.2009 | 0.5967 | small |
| duration | _numpy._util | ours | base | 0.2226 | 0.5922 | small |
| duration | nn.attention.bias | ours | base | 0.8534 | 0.5144 | negligible |
| duration | nn.attention.flex_attention | ours | base | 0.8073 | 0.4811 | negligible |
| duration | nn.modules.rnn | ours | base | 0.2226 | 0.5922 | small |
| duration | transformers.data.metrics.squad_metrics | ours | base | 0.1761 | 0.6022 | small |
| duration | transformers.pipelines.text_generation | ours | base | 0.3871 | 0.5656 | small |
| duration | transformers.tokenization_utils_base | ours | base | 0.0080 | 0.7000 | medium |
| score | _ForestDiffusion.diffusion_with_trees_class | ours | base | 1.0000 | 0.5000 | negligible |
| score | _ctgan.transformer | ours | base | 0.0000 | 0.1772 | large |
| score | _numpy._funcs | ours | base | 0.0000 | 0.1878 | large |
| score | _numpy._getlimits | ours | base | 1.0000 | 0.5000 | negligible |
| score | _numpy._ndarray | ours | base | 0.2186 | 0.5889 | small |
| score | _numpy._normalizations | ours | base | 0.0000 | 0.8689 | large |
| score | _numpy._ufuncs | ours | base | 0.5972 | 0.4844 | negligible |
| score | _numpy._util | ours | base | 0.0000 | 0.1456 | large |
| score | nn.attention.bias | ours | base | 0.0000 | 0.0450 | large |
| score | nn.attention.flex_attention | ours | base | 0.0000 | 1.0000 | large |

Continued on next page

Table 1: All statistical comparisons: classic vs ours

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|--------|---|------|------|--------|----------------|------------|
| score | nn.modules.rnn | ours | base | 0.0000 | 0.1978 | large |
| score | transformers.data.metrics.squad_metrics | ours | base | 0.6342 | 0.5344 | negligible |
| score | transformers.pipelines.text_generation | ours | base | 1.0000 | 0.5000 | negligible |
| score | transformers.tokenization_utils_base | ours | base | 0.0000 | 0.9667 | large |

B.2 Prompt comparisons

Table 2: All statistical comparisons: prompt

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|----------|---|---------------------|------------------------|--------|----------------|------------|
| duration | _ForestDiffusion.diffusion_with_trees_class | Default Prompt | NoCompressionPrompt | 0.0515 | 0.3733 | small |
| duration | _ctgan.transformer | Default Prompt | NoCompressionPrompt | 0.0000 | 0.2078 | large |
| duration | _numpy._funcs | Default Prompt | NoCompressionPrompt | 0.2358 | 0.4228 | small |
| duration | _numpy._getlimits | Default Prompt | NoCompressionPrompt | 0.0438 | 0.6311 | small |
| duration | _numpy._ndarray | Default Prompt | NoCompressionPrompt | 0.0294 | 0.6417 | medium |
| duration | _numpy._normalizations | Default Prompt | NoCompressionPrompt | 0.0036 | 0.3106 | medium |
| duration | _numpy._ufuncs | Default Prompt | NoCompressionPrompt | 0.6226 | 0.4678 | negligible |
| duration | _numpy._util | Default Prompt | NoCompressionPrompt | 0.5462 | 0.4606 | negligible |
| duration | nn.attention.bias | Default Prompt | NoCompressionPrompt | 0.5692 | 0.5372 | negligible |
| duration | nn.attention.flex_attention | Default Prompt | NoCompressionPrompt | 0.2427 | 0.5761 | small |
| duration | nn.modules.rnn | Default Prompt | NoCompressionPrompt | 0.5181 | 0.4578 | negligible |
| duration | transformers.data.metrics.squad_metrics | Default Prompt | NoCompressionPrompt | 0.0092 | 0.3306 | medium |
| duration | transformers.pipelines.text_generation | Default Prompt | NoCompressionPrompt | 0.6532 | 0.5294 | negligible |
| duration | transformers.tokenization_utils_base | Default Prompt | NoCompressionPrompt | 0.0006 | 0.7222 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | Default Prompt | NoCompressionPrompt | 0.0000 | 0.8250 | large |
| score | _ctgan.transformer | Default Prompt | NoCompressionPrompt | 0.0005 | 0.7136 | large |
| score | _numpy._funcs | Default Prompt | NoCompressionPrompt | 0.0000 | 0.8069 | large |
| score | _numpy._getlimits | Default Prompt | NoCompressionPrompt | 0.0004 | 0.6867 | medium |
| score | _numpy._ndarray | Default Prompt | NoCompressionPrompt | 0.0000 | 0.7572 | large |
| score | _numpy._normalizations | Default Prompt | NoCompressionPrompt | 0.0000 | 0.8094 | large |
| score | _numpy._ufuncs | Default Prompt | NoCompressionPrompt | 0.0000 | 0.7786 | large |
| score | _numpy._util | Default Prompt | NoCompressionPrompt | 0.0000 | 0.7608 | large |
| score | nn.attention.bias | Default Prompt | NoCompressionPrompt | 0.0000 | 0.8500 | large |
| score | nn.attention.flex_attention | Default Prompt | NoCompressionPrompt | 0.0000 | 0.7439 | large |
| score | nn.modules.rnn | Default Prompt | NoCompressionPrompt | 0.0000 | 0.7914 | large |
| score | transformers.data.metrics.squad_metrics | Default Prompt | NoCompressionPrompt | 0.0012 | 0.7061 | medium |
| score | transformers.pipelines.text_generation | Default Prompt | NoCompressionPrompt | 0.0007 | 0.6917 | medium |
| score | transformers.tokenization_utils_base | Default Prompt | NoCompressionPrompt | 0.0138 | 0.4500 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | Default Prompt | UncoveredTargetsPrompt | 0.0048 | 0.3506 | medium |
| duration | _ctgan.transformer | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.2692 | large |
| duration | _numpy._funcs | Default Prompt | UncoveredTargetsPrompt | 0.1234 | 0.4183 | small |
| duration | _numpy._getlimits | Default Prompt | UncoveredTargetsPrompt | 0.0791 | 0.5931 | small |
| duration | _numpy._ndarray | Default Prompt | UncoveredTargetsPrompt | 0.3966 | 0.4550 | negligible |
| duration | _numpy._normalizations | Default Prompt | UncoveredTargetsPrompt | 0.0036 | 0.3458 | medium |
| duration | _numpy._ufuncs | Default Prompt | UncoveredTargetsPrompt | 0.0045 | 0.3494 | medium |
| duration | _numpy._util | Default Prompt | UncoveredTargetsPrompt | 0.0039 | 0.3472 | medium |
| duration | nn.attention.bias | Default Prompt | UncoveredTargetsPrompt | 0.6015 | 0.4722 | negligible |
| duration | nn.attention.flex_attention | Default Prompt | UncoveredTargetsPrompt | 0.5726 | 0.4700 | negligible |
| duration | nn.modules.rnn | Default Prompt | UncoveredTargetsPrompt | 0.1100 | 0.4153 | small |
| duration | transformers.data.metrics.squad_metrics | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.0761 | large |
| duration | transformers.pipelines.text_generation | Default Prompt | UncoveredTargetsPrompt | 0.6842 | 0.4783 | negligible |
| duration | transformers.tokenization_utils_base | Default Prompt | UncoveredTargetsPrompt | 0.0379 | 0.3900 | small |
| score | _ForestDiffusion.diffusion_with_trees_class | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.8833 | large |
| score | _ctgan.transformer | Default Prompt | UncoveredTargetsPrompt | 0.0001 | 0.6929 | medium |
| score | _numpy._funcs | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.7689 | large |
| score | _numpy._getlimits | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.7017 | medium |
| score | _numpy._ndarray | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.7149 | large |
| score | _numpy._normalizations | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.7461 | large |
| score | _numpy._ufuncs | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.7249 | large |
| score | _numpy._util | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.8807 | large |
| score | nn.attention.bias | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.7482 | large |
| score | nn.attention.flex_attention | Default Prompt | UncoveredTargetsPrompt | 0.0003 | 0.6639 | medium |
| score | nn.modules.rnn | Default Prompt | UncoveredTargetsPrompt | 0.0406 | 0.6025 | small |
| score | transformers.data.metrics.squad_metrics | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.8750 | large |
| score | transformers.pipelines.text_generation | Default Prompt | UncoveredTargetsPrompt | 0.0000 | 0.7000 | medium |
| score | transformers.tokenization_utils_base | Default Prompt | UncoveredTargetsPrompt | 0.0002 | 0.3917 | small |
| duration | _ForestDiffusion.diffusion_with_trees_class | NoCompressionPrompt | UncoveredTargetsPrompt | 0.3399 | 0.4378 | small |
| duration | _ctgan.transformer | NoCompressionPrompt | UncoveredTargetsPrompt | 0.2751 | 0.5711 | small |
| duration | _numpy._funcs | NoCompressionPrompt | UncoveredTargetsPrompt | 0.5809 | 0.4639 | negligible |
| duration | _numpy._getlimits | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0053 | 0.3189 | medium |
| duration | _numpy._ndarray | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0038 | 0.3117 | medium |
| duration | _numpy._normalizations | NoCompressionPrompt | UncoveredTargetsPrompt | 0.5519 | 0.4611 | negligible |
| duration | _numpy._ufuncs | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0547 | 0.3750 | small |
| duration | _numpy._util | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0139 | 0.3400 | medium |
| duration | nn.attention.bias | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0032 | 0.3083 | medium |
| duration | nn.attention.flex_attention | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0027 | 0.3050 | medium |
| duration | nn.modules.rnn | NoCompressionPrompt | UncoveredTargetsPrompt | 0.2325 | 0.4222 | small |
| duration | transformers.data.metrics.squad_metrics | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0000 | 0.1561 | large |
| duration | transformers.pipelines.text_generation | NoCompressionPrompt | UncoveredTargetsPrompt | 0.1309 | 0.4017 | small |
| duration | transformers.tokenization_utils_base | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0006 | 0.2761 | large |

Continued on next page

Table 2: All statistical comparisons: prompt

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|--------|---|---------------------|------------------------|--------|----------------|------------|
| score | _ForestDiffusion.diffusion_with_trees_class | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0242 | 0.5583 | negligible |
| score | _ctgan.transformer | NoCompressionPrompt | UncoveredTargetsPrompt | 0.8008 | 0.4881 | negligible |
| score | _numpy._funcs | NoCompressionPrompt | UncoveredTargetsPrompt | 0.2558 | 0.4442 | negligible |
| score | _numpy._getlimits | NoCompressionPrompt | UncoveredTargetsPrompt | 0.4794 | 0.5167 | negligible |
| score | _numpy._ndarray | NoCompressionPrompt | UncoveredTargetsPrompt | 0.2154 | 0.4522 | negligible |
| score | _numpy._normalizations | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0322 | 0.4147 | small |
| score | _numpy._ufuncs | NoCompressionPrompt | UncoveredTargetsPrompt | 0.1344 | 0.4492 | negligible |
| score | _numpy._util | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0321 | 0.5906 | small |
| score | nn.attention.bias | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0133 | 0.4083 | small |
| score | nn.attention.flex_attention | NoCompressionPrompt | UncoveredTargetsPrompt | 0.1216 | 0.4383 | small |
| score | nn.modules.rnn | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0037 | 0.3533 | medium |
| score | transformers.data.metrics.squad_metrics | NoCompressionPrompt | UncoveredTargetsPrompt | 0.0502 | 0.5961 | small |
| score | transformers.pipelines.text_generation | NoCompressionPrompt | UncoveredTargetsPrompt | 0.8695 | 0.5083 | negligible |
| score | transformers.tokenization_utils_base | NoCompressionPrompt | UncoveredTargetsPrompt | 0.1296 | 0.4342 | small |

B.3 Model comparisons

Table 3: All statistical comparisons: coder vs non coder

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|----------|---|--------------------|--------------|--------|----------------|------------|
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:3b | qwen2.5:3b | 0.5895 | 0.5411 | negligible |
| duration | _ctgan.transformer | qwen2.5-coder:3b | qwen2.5:3b | 0.1669 | 0.3956 | small |
| duration | _numpy._getlimits | qwen2.5-coder:3b | qwen2.5:3b | 0.7845 | 0.5211 | negligible |
| duration | _numpy._ndarray | qwen2.5-coder:3b | qwen2.5:3b | 0.5298 | 0.4522 | negligible |
| duration | nn.attention.bias | qwen2.5-coder:3b | qwen2.5:3b | 0.1154 | 0.6189 | small |
| duration | nn.attention.flex_attention | qwen2.5-coder:3b | qwen2.5:3b | 0.1297 | 0.6144 | small |
| duration | nn.modules.rnn | qwen2.5-coder:3b | qwen2.5:3b | 0.7731 | 0.4778 | negligible |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:3b | qwen2.5:3b | 0.0127 | 0.6878 | medium |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:3b | qwen2.5:3b | 0.6952 | 0.5300 | negligible |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:3b | qwen2.5:3b | 0.7958 | 0.4800 | negligible |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:3b | qwen2.5:3b | 0.3128 | 0.4667 | negligible |
| score | _ctgan.transformer | qwen2.5-coder:3b | qwen2.5:3b | 0.9364 | 0.4956 | negligible |
| score | _numpy._getlimits | qwen2.5-coder:3b | qwen2.5:3b | 0.1694 | 0.5500 | negligible |
| score | _numpy._ndarray | qwen2.5-coder:3b | qwen2.5:3b | 0.7836 | 0.4794 | negligible |
| score | nn.attention.bias | qwen2.5-coder:3b | qwen2.5:3b | 0.0109 | 0.6000 | small |
| score | nn.attention.flex_attention | qwen2.5-coder:3b | qwen2.5:3b | 0.0059 | 0.3194 | medium |
| score | nn.modules.rnn | qwen2.5-coder:3b | qwen2.5:3b | 0.1394 | 0.5922 | small |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:3b | qwen2.5:3b | 0.0216 | 0.3411 | medium |
| score | transformers.pipelines.text_generation | qwen2.5-coder:3b | qwen2.5:3b | 0.3337 | 0.5167 | negligible |
| score | transformers.tokenization_utils_base | qwen2.5-coder:3b | qwen2.5:3b | 0.5440 | 0.4822 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.2458 | 0.4122 | small |
| duration | _ctgan.transformer | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0555 | 0.3556 | medium |
| duration | _numpy._getlimits | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.9352 | 0.5067 | negligible |
| duration | _numpy._ndarray | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.1224 | 0.3833 | small |
| duration | nn.attention.bias | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.1055 | 0.6222 | small |
| duration | nn.attention.flex_attention | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.5298 | 0.5478 | negligible |
| duration | nn.modules.rnn | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.1669 | 0.6044 | small |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0076 | 0.7011 | medium |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.4733 | 0.5544 | negligible |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0798 | 0.3678 | small |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.1607 | 0.5333 | negligible |
| score | _ctgan.transformer | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0980 | 0.5778 | small |
| score | _numpy._getlimits | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0419 | 0.5667 | small |
| score | _numpy._ndarray | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.7668 | 0.5161 | negligible |
| score | nn.attention.bias | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.7477 | 0.5156 | negligible |
| score | nn.attention.flex_attention | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.7502 | 0.5139 | negligible |
| score | nn.modules.rnn | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.7507 | 0.4833 | negligible |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0043 | 0.3406 | medium |
| score | transformers.pipelines.text_generation | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0750 | 0.5833 | small |
| score | transformers.tokenization_utils_base | qwen2.5-coder:1.5b | qwen2.5:1.5b | 0.0782 | 0.5828 | small |

Table 4: All statistical comparisons: model weight

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|----------|---|------------|------------|--------|----------------|------------|
| duration | _ForestDiffusion.diffusion_with_trees_class | gemma4:e2b | gemma4:12b | 0.0010 | 0.7478 | large |
| duration | _ctgan.transformer | gemma4:e2b | gemma4:12b | 0.0000 | 0.8944 | large |
| duration | _numpy._getlimits | gemma4:e2b | gemma4:12b | 0.0215 | 0.6733 | medium |
| duration | _numpy._ndarray | gemma4:e2b | gemma4:12b | 0.0000 | 0.9044 | large |
| duration | nn.attention.bias | gemma4:e2b | gemma4:12b | 0.2282 | 0.5911 | small |
| duration | nn.attention.flex_attention | gemma4:e2b | gemma4:12b | 0.0933 | 0.6267 | small |
| duration | nn.modules.rnn | gemma4:e2b | gemma4:12b | 0.0199 | 0.6756 | medium |
| duration | transformers.data.metrics.squad_metrics | gemma4:e2b | gemma4:12b | 0.0000 | 0.9678 | large |
| duration | transformers.pipelines.text_generation | gemma4:e2b | gemma4:12b | 0.0009 | 0.7500 | large |
| duration | transformers.tokenization_utils_base | gemma4:e2b | gemma4:12b | 0.0000 | 0.9522 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | gemma4:e2b | gemma4:12b | 1.0000 | 0.5000 | negligible |
| score | _ctgan.transformer | gemma4:e2b | gemma4:12b | 0.0419 | 0.5667 | small |

Continued on next page

Table 4: All statistical comparisons: model weight

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|----------|---|------------|------------|--------|----------------|------------|
| score | _numpy._getlimits | gemma4:e2b | gemma4:12b | 0.0000 | 0.2333 | large |
| score | _numpy._ndarray | gemma4:e2b | gemma4:12b | 0.0019 | 0.2817 | large |
| score | nn.attention.bias | gemma4:e2b | gemma4:12b | 0.3337 | 0.5167 | negligible |
| score | nn.attention.flex_attention | gemma4:e2b | gemma4:12b | 0.3337 | 0.5167 | negligible |
| score | nn.modules.rnn | gemma4:e2b | gemma4:12b | 0.0026 | 0.6533 | medium |
| score | transformers.data.metrics.squad_metrics | gemma4:e2b | gemma4:12b | 1.0000 | 0.4994 | negligible |
| score | transformers.pipelines.text_generation | gemma4:e2b | gemma4:12b | 0.3337 | 0.4833 | negligible |
| score | transformers.tokenization_utils_base | gemma4:e2b | gemma4:12b | 0.1797 | 0.4511 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | gemma4:e2b | gemma4:26b | 0.0406 | 0.6544 | medium |
| duration | _ctgan.transformer | gemma4:e2b | gemma4:26b | 0.0000 | 0.9011 | large |
| duration | _numpy._getlimits | gemma4:e2b | gemma4:26b | 0.0378 | 0.6567 | medium |
| duration | _numpy._ndarray | gemma4:e2b | gemma4:26b | 0.2905 | 0.5800 | small |
| duration | nn.attention.bias | gemma4:e2b | gemma4:26b | 0.3871 | 0.5656 | small |
| duration | nn.attention.flex_attention | gemma4:e2b | gemma4:26b | 0.0963 | 0.6256 | small |
| duration | nn.modules.rnn | gemma4:e2b | gemma4:26b | 0.0176 | 0.6789 | medium |
| duration | transformers.data.metrics.squad_metrics | gemma4:e2b | gemma4:26b | 0.0000 | 0.9522 | large |
| duration | transformers.pipelines.text_generation | gemma4:e2b | gemma4:26b | 0.0009 | 0.7500 | large |
| duration | transformers.tokenization_utils_base | gemma4:e2b | gemma4:26b | 0.0000 | 0.9400 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | gemma4:e2b | gemma4:26b | 0.3337 | 0.5167 | negligible |
| score | _ctgan.transformer | gemma4:e2b | gemma4:26b | 0.6089 | 0.4756 | negligible |
| score | _numpy._getlimits | gemma4:e2b | gemma4:26b | 0.0905 | 0.4333 | small |
| score | _numpy._ndarray | gemma4:e2b | gemma4:26b | 0.3082 | 0.4322 | small |
| score | nn.attention.bias | gemma4:e2b | gemma4:26b | 0.3337 | 0.5167 | negligible |
| score | nn.attention.flex_attention | gemma4:e2b | gemma4:26b | 1.0000 | 0.4994 | negligible |
| score | nn.modules.rnn | gemma4:e2b | gemma4:26b | 0.9046 | 0.4917 | negligible |
| score | transformers.data.metrics.squad_metrics | gemma4:e2b | gemma4:26b | 0.1608 | 0.5333 | negligible |
| score | transformers.pipelines.text_generation | gemma4:e2b | gemma4:26b | 0.0418 | 0.4333 | small |
| score | transformers.tokenization_utils_base | gemma4:e2b | gemma4:26b | 0.5972 | 0.4844 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | gemma4:e2b | gemma4:31b | 0.0004 | 0.7689 | large |
| duration | _ctgan.transformer | gemma4:e2b | gemma4:31b | 0.0000 | 0.9378 | large |
| duration | _numpy._getlimits | gemma4:e2b | gemma4:31b | 0.0364 | 0.6578 | medium |
| duration | _numpy._ndarray | gemma4:e2b | gemma4:31b | 0.0000 | 0.8278 | large |
| duration | nn.attention.bias | gemma4:e2b | gemma4:31b | 0.5997 | 0.5400 | negligible |
| duration | nn.attention.flex_attention | gemma4:e2b | gemma4:31b | 0.1494 | 0.6089 | small |
| duration | nn.modules.rnn | gemma4:e2b | gemma4:31b | 0.0009 | 0.7500 | large |
| duration | transformers.data.metrics.squad_metrics | gemma4:e2b | gemma4:31b | 0.0000 | 0.9678 | large |
| duration | transformers.pipelines.text_generation | gemma4:e2b | gemma4:31b | 0.0009 | 0.7500 | large |
| duration | transformers.tokenization_utils_base | gemma4:e2b | gemma4:31b | 0.0000 | 0.9678 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | gemma4:e2b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| score | _ctgan.transformer | gemma4:e2b | gemma4:31b | 0.1899 | 0.5478 | negligible |
| score | _numpy._getlimits | gemma4:e2b | gemma4:31b | 0.0007 | 0.3167 | medium |
| score | _numpy._ndarray | gemma4:e2b | gemma4:31b | 0.0003 | 0.6833 | medium |
| score | nn.attention.bias | gemma4:e2b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| score | nn.attention.flex_attention | gemma4:e2b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| score | nn.modules.rnn | gemma4:e2b | gemma4:31b | 0.0007 | 0.6667 | medium |
| score | transformers.data.metrics.squad_metrics | gemma4:e2b | gemma4:31b | 0.1608 | 0.5333 | negligible |
| score | transformers.pipelines.text_generation | gemma4:e2b | gemma4:31b | 1.0000 | 0.5000 | negligible |
| score | transformers.tokenization_utils_base | gemma4:e2b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | gemma4:12b | gemma4:26b | 0.0001 | 0.2044 | large |
| duration | _ctgan.transformer | gemma4:12b | gemma4:26b | 0.7845 | 0.5211 | negligible |
| duration | _numpy._getlimits | gemma4:12b | gemma4:26b | 0.0016 | 0.2622 | large |
| duration | _numpy._ndarray | gemma4:12b | gemma4:26b | 0.0001 | 0.2056 | large |
| duration | nn.attention.bias | gemma4:12b | gemma4:26b | 0.3478 | 0.4289 | small |
| duration | nn.attention.flex_attention | gemma4:12b | gemma4:26b | 0.0127 | 0.3122 | medium |
| duration | nn.modules.rnn | gemma4:12b | gemma4:26b | 0.6952 | 0.5300 | negligible |
| duration | transformers.data.metrics.squad_metrics | gemma4:12b | gemma4:26b | 0.0000 | 0.0467 | large |
| duration | transformers.pipelines.text_generation | gemma4:12b | gemma4:26b | 0.0061 | 0.2933 | medium |
| duration | transformers.tokenization_utils_base | gemma4:12b | gemma4:26b | 0.0061 | 0.2933 | medium |
| score | _ForestDiffusion.diffusion_with_trees_class | gemma4:12b | gemma4:26b | 0.3337 | 0.5167 | negligible |
| score | _ctgan.transformer | gemma4:12b | gemma4:26b | 0.0216 | 0.4167 | small |
| score | _numpy._getlimits | gemma4:12b | gemma4:26b | 0.0015 | 0.7000 | medium |
| score | _numpy._ndarray | gemma4:12b | gemma4:26b | 0.1164 | 0.6128 | small |
| score | nn.attention.bias | gemma4:12b | gemma4:26b | 1.0000 | 0.5000 | negligible |
| score | nn.attention.flex_attention | gemma4:12b | gemma4:26b | 0.3337 | 0.4833 | negligible |
| score | nn.modules.rnn | gemma4:12b | gemma4:26b | 0.0008 | 0.3172 | medium |
| score | transformers.data.metrics.squad_metrics | gemma4:12b | gemma4:26b | 0.1608 | 0.5333 | negligible |
| score | transformers.pipelines.text_generation | gemma4:12b | gemma4:26b | 0.1694 | 0.4500 | negligible |
| score | transformers.tokenization_utils_base | gemma4:12b | gemma4:26b | 0.3560 | 0.5367 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | gemma4:12b | gemma4:31b | 0.0000 | 0.8278 | large |
| duration | _ctgan.transformer | gemma4:12b | gemma4:31b | 0.0000 | 0.9078 | large |
| duration | _numpy._getlimits | gemma4:12b | gemma4:31b | 0.0271 | 0.6667 | medium |
| duration | _numpy._ndarray | gemma4:12b | gemma4:31b | 0.4204 | 0.4389 | small |
| duration | nn.attention.bias | gemma4:12b | gemma4:31b | 0.0076 | 0.7011 | medium |
| duration | nn.attention.flex_attention | gemma4:12b | gemma4:31b | 0.0012 | 0.7444 | large |
| duration | nn.modules.rnn | gemma4:12b | gemma4:31b | 0.0000 | 0.8278 | large |
| duration | transformers.data.metrics.squad_metrics | gemma4:12b | gemma4:31b | 0.2116 | 0.5944 | small |
| duration | transformers.pipelines.text_generation | gemma4:12b | gemma4:31b | 0.0000 | 0.8556 | large |
| duration | transformers.tokenization_utils_base | gemma4:12b | gemma4:31b | 0.0303 | 0.6633 | medium |
| score | _ForestDiffusion.diffusion_with_trees_class | gemma4:12b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| score | _ctgan.transformer | gemma4:12b | gemma4:31b | 0.3337 | 0.4833 | negligible |
| score | _numpy._getlimits | gemma4:12b | gemma4:31b | 0.2032 | 0.5833 | small |
| score | _numpy._ndarray | gemma4:12b | gemma4:31b | 0.0000 | 0.8500 | large |
| score | nn.attention.bias | gemma4:12b | gemma4:31b | 1.0000 | 0.5000 | negligible |
| score | nn.attention.flex_attention | gemma4:12b | gemma4:31b | 1.0000 | 0.5000 | negligible |

Continued on next page

Table 4: All statistical comparisons: model weight

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|----------|---|--------------------|--------------------|--------|----------------|------------|
| score | nn.modules.rnn | gemma4:12b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| score | transformers.data.metrics.squad_metrics | gemma4:12b | gemma4:31b | 0.1608 | 0.5333 | negligible |
| score | transformers.pipelines.text_generation | gemma4:12b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| score | transformers.tokenization_utils_base | gemma4:12b | gemma4:31b | 0.0419 | 0.5667 | small |
| duration | _ForestDiffusion.diffusion_with_trees_class | gemma4:26b | gemma4:31b | 0.0000 | 0.8844 | large |
| duration | _ctgan.transformer | gemma4:26b | gemma4:31b | 0.0000 | 0.8367 | large |
| duration | _numpy._getlimits | gemma4:26b | gemma4:31b | 0.0001 | 0.7989 | large |
| duration | _numpy._ndarray | gemma4:26b | gemma4:31b | 0.0042 | 0.7156 | large |
| duration | nn.attention.bias | gemma4:26b | gemma4:31b | 0.0015 | 0.7389 | large |
| duration | nn.attention.flex_attention | gemma4:26b | gemma4:31b | 0.0002 | 0.7767 | large |
| duration | nn.modules.rnn | gemma4:26b | gemma4:31b | 0.0232 | 0.6711 | medium |
| duration | transformers.data.metrics.squad_metrics | gemma4:26b | gemma4:31b | 0.0000 | 1.0000 | large |
| duration | transformers.pipelines.text_generation | gemma4:26b | gemma4:31b | 0.0000 | 0.8589 | large |
| duration | transformers.tokenization_utils_base | gemma4:26b | gemma4:31b | 0.0000 | 0.9411 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | gemma4:26b | gemma4:31b | 1.0000 | 0.5000 | negligible |
| score | _ctgan.transformer | gemma4:26b | gemma4:31b | 0.0966 | 0.5656 | small |
| score | _numpy._getlimits | gemma4:26b | gemma4:31b | 0.0478 | 0.3833 | small |
| score | _numpy._ndarray | gemma4:26b | gemma4:31b | 0.0001 | 0.7167 | large |
| score | nn.attention.bias | gemma4:26b | gemma4:31b | 1.0000 | 0.5000 | negligible |
| score | nn.attention.flex_attention | gemma4:26b | gemma4:31b | 0.3337 | 0.5167 | negligible |
| score | nn.modules.rnn | gemma4:26b | gemma4:31b | 0.0001 | 0.7000 | medium |
| score | transformers.data.metrics.squad_metrics | gemma4:26b | gemma4:31b | 1.0000 | 0.5000 | negligible |
| score | transformers.pipelines.text_generation | gemma4:26b | gemma4:31b | 0.0418 | 0.5667 | small |
| score | transformers.tokenization_utils_base | gemma4:26b | gemma4:31b | 0.1608 | 0.5333 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.8883 | 0.4889 | negligible |
| duration | _ctgan.transformer | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.1958 | 0.5978 | small |
| duration | _numpy._getlimits | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.0679 | 0.6378 | small |
| duration | _numpy._ndarray | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.2581 | 0.5856 | small |
| duration | nn.attention.bias | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.3112 | 0.5767 | small |
| duration | nn.attention.flex_attention | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.7283 | 0.4733 | negligible |
| duration | nn.modules.rnn | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.2009 | 0.4033 | small |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.2458 | 0.4122 | small |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.5793 | 0.4578 | negligible |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.8883 | 0.5111 | negligible |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.0750 | 0.5833 | small |
| score | _ctgan.transformer | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.7799 | 0.5161 | negligible |
| score | _numpy._getlimits | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.7119 | 0.5178 | negligible |
| score | _numpy._ndarray | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.1065 | 0.4278 | small |
| score | nn.attention.bias | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.7120 | 0.4822 | negligible |
| score | nn.attention.flex_attention | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.3666 | 0.5461 | negligible |
| score | nn.modules.rnn | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.5606 | 0.5311 | negligible |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.0018 | 0.6789 | medium |
| score | transformers.pipelines.text_generation | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.5291 | 0.4667 | negligible |
| score | transformers.tokenization_utils_base | qwen2.5-coder:0.5b | qwen2.5-coder:1.5b | 0.2780 | 0.4444 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.7283 | 0.5267 | negligible |
| duration | _ctgan.transformer | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.0133 | 0.6867 | medium |
| duration | _numpy._getlimits | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.1580 | 0.6067 | small |
| duration | _numpy._ndarray | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.1120 | 0.6200 | small |
| duration | nn.attention.bias | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.9117 | 0.4911 | negligible |
| duration | nn.attention.flex_attention | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.9234 | 0.4922 | negligible |
| duration | nn.modules.rnn | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.5493 | 0.5456 | negligible |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.0292 | 0.6644 | medium |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.9941 | 0.5011 | negligible |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.1023 | 0.6233 | small |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.0246 | 0.6000 | small |
| score | _ctgan.transformer | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.4397 | 0.5411 | negligible |
| score | _numpy._getlimits | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.8023 | 0.5122 | negligible |
| score | _numpy._ndarray | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.0000 | 0.2200 | large |
| score | nn.attention.bias | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.5914 | 0.4733 | negligible |
| score | nn.attention.flex_attention | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.9343 | 0.5050 | negligible |
| score | nn.modules.rnn | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.0541 | 0.3800 | small |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.3482 | 0.5606 | small |
| score | transformers.pipelines.text_generation | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.0905 | 0.5667 | small |
| score | transformers.tokenization_utils_base | qwen2.5-coder:0.5b | qwen2.5-coder:3b | 0.1797 | 0.5489 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.4643 | 0.4444 | negligible |
| duration | _ctgan.transformer | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0392 | 0.6556 | medium |
| duration | _numpy._getlimits | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.6952 | 0.5300 | negligible |
| duration | _numpy._ndarray | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.6952 | 0.4700 | negligible |
| duration | nn.attention.bias | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.4035 | 0.4367 | small |
| duration | nn.attention.flex_attention | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.3112 | 0.4233 | small |
| duration | nn.modules.rnn | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0351 | 0.6589 | medium |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0012 | 0.7433 | large |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.9117 | 0.4911 | negligible |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0011 | 0.7467 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0246 | 0.6000 | small |
| score | _ctgan.transformer | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.4779 | 0.5378 | negligible |
| score | _numpy._getlimits | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.9181 | 0.4944 | negligible |
| score | _numpy._ndarray | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0000 | 0.2211 | large |
| score | nn.attention.bias | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0419 | 0.5667 | small |
| score | nn.attention.flex_attention | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.9343 | 0.5050 | negligible |
| score | nn.modules.rnn | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.0099 | 0.3344 | medium |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.6976 | 0.4733 | negligible |
| score | transformers.pipelines.text_generation | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.7289 | 0.5167 | negligible |
| score | transformers.tokenization_utils_base | qwen2.5-coder:0.5b | qwen2.5-coder:7b | 0.3930 | 0.4578 | negligible |

Continued on next page

Table 4: All statistical comparisons: model weight

| Metric | Sample | A | B | p | $\hat{A}_{12}$ | Effect |
|----------|---|--------------------|------------------|--------|----------------|------------|
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.4204 | 0.5611 | small |
| duration | _ctgan.transformer | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.1413 | 0.6111 | small |
| duration | _numpy._getlimits | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0292 | 0.6644 | medium |
| duration | _numpy._ndarray | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.6843 | 0.4689 | negligible |
| duration | nn.attention.bias | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0001 | 0.8022 | large |
| duration | nn.attention.flex_attention | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0099 | 0.6944 | medium |
| duration | nn.modules.rnn | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0850 | 0.6300 | small |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0000 | 0.8433 | large |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.1624 | 0.6056 | small |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0537 | 0.6456 | medium |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.5702 | 0.5167 | negligible |
| score | _ctgan.transformer | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.5817 | 0.5294 | negligible |
| score | _numpy._getlimits | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.9301 | 0.4956 | negligible |
| score | _numpy._ndarray | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0085 | 0.3244 | medium |
| score | nn.attention.bias | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.8517 | 0.4900 | negligible |
| score | nn.attention.flex_attention | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.4518 | 0.4628 | negligible |
| score | nn.modules.rnn | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0196 | 0.3600 | medium |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0425 | 0.4006 | small |
| score | transformers.pipelines.text_generation | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0246 | 0.6000 | small |
| score | transformers.tokenization_utils_base | qwen2.5-coder:1.5b | qwen2.5-coder:3b | 0.0243 | 0.6006 | small |
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.7283 | 0.4733 | negligible |
| duration | _ctgan.transformer | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.5298 | 0.5478 | negligible |
| duration | _numpy._getlimits | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.2643 | 0.5844 | small |
| duration | _numpy._ndarray | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0091 | 0.3033 | medium |
| duration | nn.attention.bias | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0421 | 0.6533 | medium |
| duration | nn.attention.flex_attention | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.2458 | 0.5878 | small |
| duration | nn.modules.rnn | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0083 | 0.6989 | medium |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0000 | 0.8978 | large |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.1297 | 0.6144 | small |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0003 | 0.7744 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.5702 | 0.5167 | negligible |
| score | _ctgan.transformer | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.6717 | 0.5228 | negligible |
| score | _numpy._getlimits | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.6419 | 0.4778 | negligible |
| score | _numpy._ndarray | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0066 | 0.3189 | medium |
| score | nn.attention.bias | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0215 | 0.5833 | small |
| score | nn.attention.flex_attention | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.4518 | 0.4628 | negligible |
| score | nn.modules.rnn | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0024 | 0.3111 | medium |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.0025 | 0.3311 | medium |
| score | transformers.pipelines.text_generation | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.3263 | 0.5500 | negligible |
| score | transformers.tokenization_utils_base | qwen2.5-coder:1.5b | qwen2.5-coder:7b | 0.8215 | 0.5128 | negligible |
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.4204 | 0.4389 | small |
| duration | _ctgan.transformer | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.4290 | 0.4400 | small |
| duration | _numpy._getlimits | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.2905 | 0.5800 | small |
| duration | _numpy._ndarray | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.0056 | 0.2911 | medium |
| duration | nn.attention.bias | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.0040 | 0.7167 | large |
| duration | nn.attention.flex_attention | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.3403 | 0.5722 | small |
| duration | nn.modules.rnn | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.1453 | 0.6100 | small |
| duration | transformers.data.metrics.squad_metrics | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.0080 | 0.7000 | medium |
| duration | transformers.pipelines.text_generation | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.2707 | 0.5833 | small |
| duration | transformers.tokenization_utils_base | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.0108 | 0.6922 | medium |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5-coder:3b | qwen2.5-coder:7b | 1.0000 | 0.5000 | negligible |
| score | _ctgan.transformer | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.9364 | 0.4956 | negligible |
| score | _numpy._getlimits | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.7289 | 0.4833 | negligible |
| score | _numpy._ndarray | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.9877 | 0.4983 | negligible |
| score | nn.attention.bias | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.0109 | 0.6000 | small |
| score | nn.attention.flex_attention | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.9915 | 0.4989 | negligible |
| score | nn.modules.rnn | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.3514 | 0.4361 | small |
| score | transformers.data.metrics.squad_metrics | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.2055 | 0.4200 | small |
| score | transformers.pipelines.text_generation | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.1694 | 0.4500 | negligible |
| score | transformers.tokenization_utils_base | qwen2.5-coder:3b | qwen2.5-coder:7b | 0.0425 | 0.4144 | small |
| duration | _ForestDiffusion.diffusion_with_trees_class | qwen2.5:1.5b | qwen2.5:3b | 0.0122 | 0.6889 | medium |
| duration | _ctgan.transformer | qwen2.5:1.5b | qwen2.5:3b | 0.0615 | 0.6411 | medium |
| duration | _numpy._getlimits | qwen2.5:1.5b | qwen2.5:3b | 0.0575 | 0.6433 | medium |
| duration | _numpy._ndarray | qwen2.5:1.5b | qwen2.5:3b | 0.5201 | 0.5489 | negligible |
| duration | nn.attention.bias | qwen2.5:1.5b | qwen2.5:3b | 0.0378 | 0.6567 | medium |
| duration | nn.attention.flex_attention | qwen2.5:1.5b | qwen2.5:3b | 0.0232 | 0.6711 | medium |
| duration | nn.modules.rnn | qwen2.5:1.5b | qwen2.5:3b | 0.4290 | 0.5600 | small |
| duration | transformers.data.metrics.squad_metrics | qwen2.5:1.5b | qwen2.5:3b | 0.0001 | 0.8011 | large |
| duration | transformers.pipelines.text_generation | qwen2.5:1.5b | qwen2.5:3b | 0.3183 | 0.5756 | small |
| duration | transformers.tokenization_utils_base | qwen2.5:1.5b | qwen2.5:3b | 0.0002 | 0.7789 | large |
| score | _ForestDiffusion.diffusion_with_trees_class | qwen2.5:1.5b | qwen2.5:3b | 0.0814 | 0.4500 | negligible |
| score | _ctgan.transformer | qwen2.5:1.5b | qwen2.5:3b | 0.2486 | 0.4511 | negligible |
| score | _numpy._getlimits | qwen2.5:1.5b | qwen2.5:3b | 0.3337 | 0.4833 | negligible |
| score | _numpy._ndarray | qwen2.5:1.5b | qwen2.5:3b | 0.0000 | 0.1828 | large |
| score | nn.attention.bias | qwen2.5:1.5b | qwen2.5:3b | 0.0419 | 0.5667 | small |
| score | nn.attention.flex_attention | qwen2.5:1.5b | qwen2.5:3b | 0.0002 | 0.2689 | large |
| score | nn.modules.rnn | qwen2.5:1.5b | qwen2.5:3b | 0.6666 | 0.4761 | negligible |
| score | transformers.data.metrics.squad_metrics | qwen2.5:1.5b | qwen2.5:3b | 0.0090 | 0.3139 | medium |
| score | transformers.pipelines.text_generation | qwen2.5:1.5b | qwen2.5:3b | 0.1607 | 0.5333 | negligible |
| score | transformers.tokenization_utils_base | qwen2.5:1.5b | qwen2.5:3b | 1.0000 | 0.5000 | negligible |

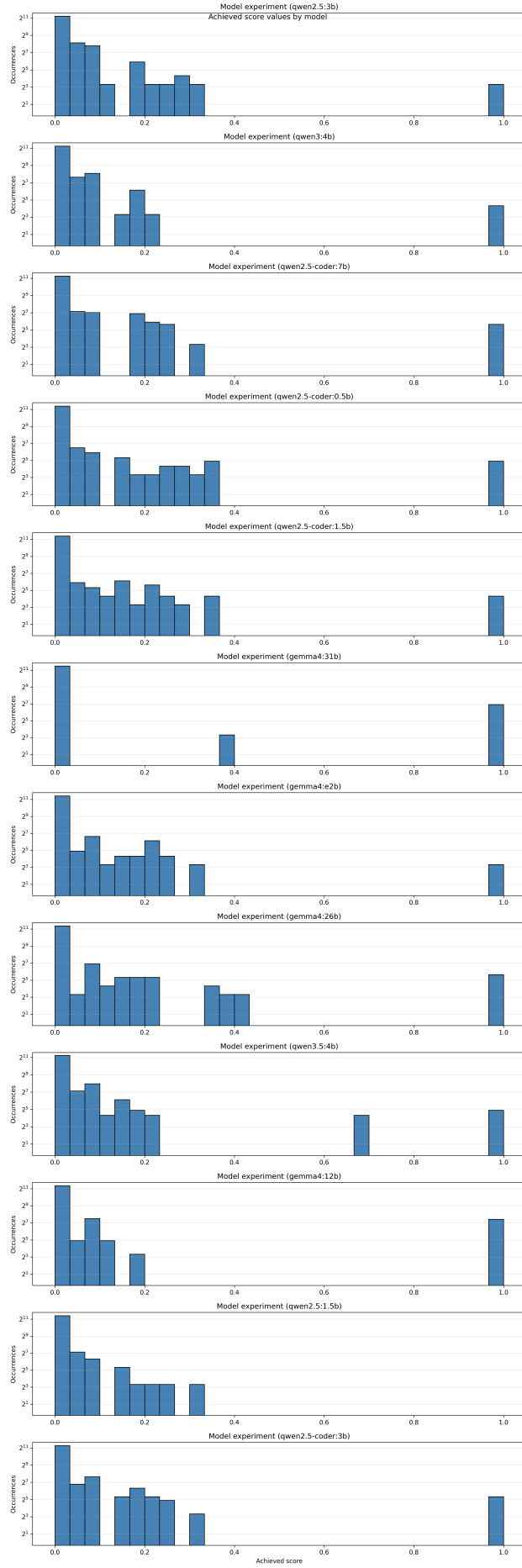


Figure 8: Distribution of coverage scores across different LLM models (*log scale*)